

IC526 Advanced Compilers

Lec 08: Type Checker I

Yoonseung Kim

DGIST EECS

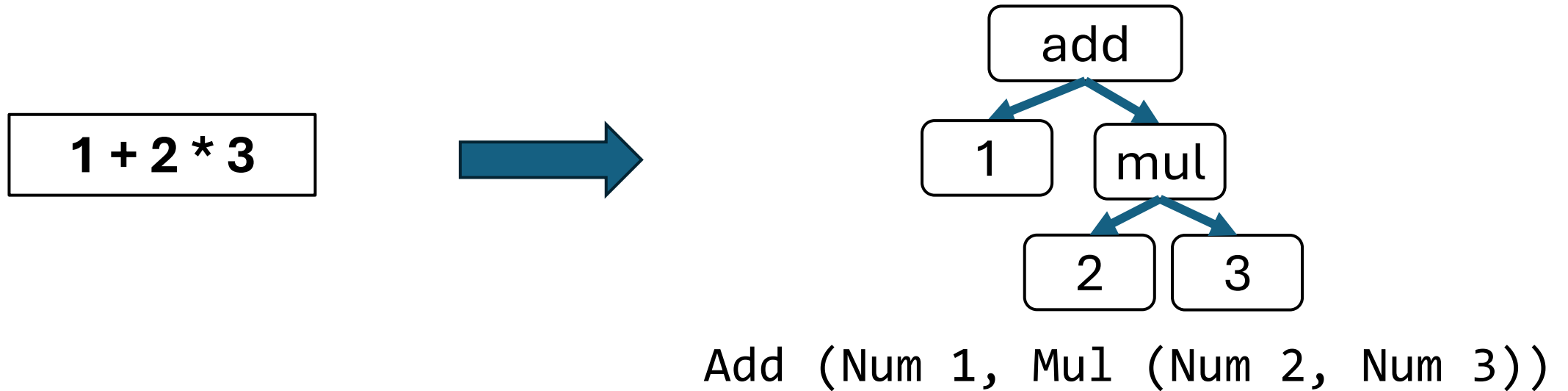
Fall 2026

Announcement

- Programming Assignment 2 is out.
 - Lexer & Parser
 - Deadline : Sep 27th

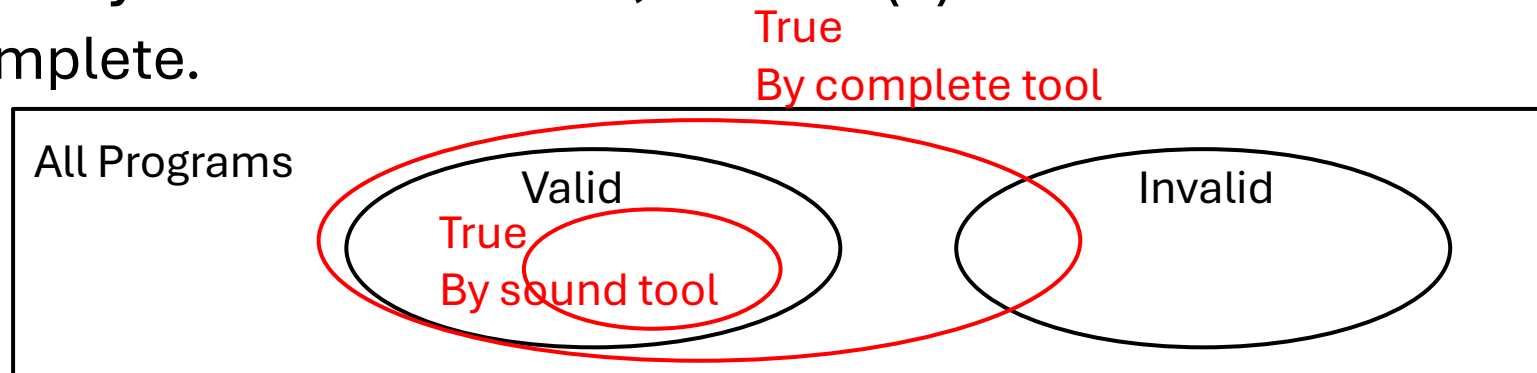
Recap: Parsers and AST

- Parsers construct a tree structure from program text



Preliminary: Soundness & Completeness

- Tool and Truth
 - Suppose you are interested in proving a proposition $P(x)$.
 - E.g, $P(x)$ = “Program x is a valid program.”
 - You design a checker C that tests a program.
 - $C(p1) = \text{true}$ / $C(p2) = \text{false}$
- Good property #1: “If $C(x)$ returns ‘true’, then x is valid.”
→ C is sound.
- Good property #2: “If x is valid, then $C(x)$ returns ‘true’.”
→ C is complete.

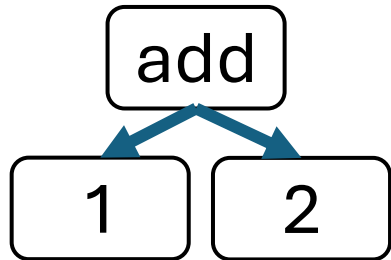


Why Type Checkers?

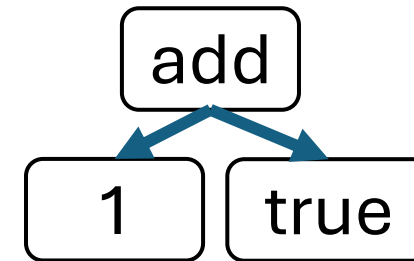
Valid Syntax, Invalid Programs

$e ::= n \mid \text{true} \mid \text{false} \mid x$
 $\mid e_1 + e_2 \mid e_1 \&\& e_2 \mid e_1 == e_2$

// integer addition
1 + 2



// the parser can also build this AST
1 + true



A typechecker detects such an
ill-formed program **before execution**.

Type Checking Uses Context

(Pseudocode with Rust-like Syntax)

Accepted

```
let x : int = 10;
```

Rejected

```
let x : bool = true;
```

```
fib(x + 1)
```

- The expression is the same in both cases.
- The **context** supplies the type of x.
- Recall: the *syntax analysis* is context-free.

Typing Judgment

Typing Judgment: A Proposition

```
fn f(a : bool) -> int { ... }
```

```
fn g(x : int) {  
  let y : bool = false;  
  let z : int = x + f(y);  
}
```

context : types of variables in the scope
 $\Gamma = \{ x : \text{int}, y : \text{bool}, f : \text{bool} \rightarrow \text{int} \}$

Today, we are interested in proving propositions like:

“In the context Γ , the expression $x + f(y)$ has type int ”

$$\Gamma \vdash x + f(y) : \text{int}$$

Typing Judgment: A Proof Goal

$\Gamma \vdash e : \tau$

Γ : a typing context

e : an expression

τ : a type

$\Gamma = \{ x : \text{int}, \text{ready} : \text{bool} \}$

$\Gamma \vdash x + 1 : \text{int} \quad \Gamma \vdash x + 1 : \text{bool}$

Q: Which judgment is correct? Which is not?

Prove It With Typing Rules

Goal: $\Gamma \vdash x + 1 : \text{int}$

- **Typing rules** specify conditions for a judgment to be true.
- The rules are language-specific
 - must be sound with respect to the language.

Example: A Small Expression Language

$\tau ::= \text{int} \mid \text{bool} \mid \tau_1 \rightarrow \tau_2 \text{ where } \tau_1, \tau_2 \in \{ \text{int}, \text{bool} \}$

$e ::= n \mid \text{true} \mid \text{false} \mid x$
 $\mid e_1 + e_2 \mid e_1 \ \&\& \ e_2 \mid e_1 == e_2 \mid e_1(e_2)$

- Addition uses integers. Conjunction uses booleans.
- Equality compares two integers or two booleans.
- A function type specifies one parameter type and a result type.

Literal Rules

Premises are above each line. The conclusion is below it.

$$\Gamma \vdash n : \text{int}$$

$$\Gamma \vdash \text{true} : \text{bool}$$

$$\Gamma \vdash \text{false} : \text{bool}$$

- These rules have no premises.
- Literal typing does not depend on Γ .

The Variable Rule

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$

$$\Gamma = \{ x : \text{int}, \text{ready} : \text{bool} \}$$

$$\Gamma(x) = \text{int}$$

$$\Gamma(\text{ready}) = \text{bool}$$

$$\Gamma(y) = \perp$$

- Look up the binding of x in Γ .
- Use the type provided by that binding.
- An unbound name has no justification from this rule.

Inductive Cases: Rules for Operations

$$\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}$$

ADD

$$\Gamma \vdash e_1 + e_2 : \text{int}$$
$$\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \text{bool}$$

AND

$$\Gamma \vdash e_1 \ \&\& \ e_2 : \text{bool}$$
$$\Gamma \vdash e_1 : \sigma \quad \Gamma \vdash e_2 : \sigma \quad \sigma \in \{\text{int}, \text{bool}\}$$

EQ

$$\Gamma \vdash e_1 == e_2 : \text{bool}$$

The Function Application Rule

$$\frac{\Gamma \vdash e_1 : \sigma \rightarrow \tau \quad \Gamma \vdash e_2 : \sigma \quad \sigma, \tau \in \{\text{int}, \text{bool}\}}{\Gamma \vdash e_1(e_2) : \tau} \text{APP}$$

$\Gamma = \{ \text{foo} : \text{bool} \rightarrow \text{int}, y : \text{bool} \}$

$\Gamma \vdash \text{foo}(y) : \text{int}$

- The argument must match the function's parameter type.
- The application has the function's result type.

All Typing Rules

$$\frac{}{\Gamma \vdash n : \text{int}}$$
$$\frac{}{\Gamma \vdash \text{true} : \text{bool}}$$
$$\frac{}{\Gamma \vdash \text{false} : \text{bool}}$$
$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$
$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}}$$
$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 \ \&\& \ e_2 : \text{bool}}$$
$$\frac{\Gamma \vdash e_1 : \sigma \quad \Gamma \vdash e_2 : \sigma \quad \sigma \in \{\text{int}, \text{bool}\}}{\Gamma \vdash e_1 == e_2 : \text{bool}}$$
$$\frac{\Gamma \vdash e_1 : \sigma \rightarrow \tau \quad \Gamma \vdash e_2 : \sigma \quad \sigma, \tau \in \{\text{int}, \text{bool}\}}{\Gamma \vdash e_1(e_2) : \tau}$$

A Complete Typing Derivation

$\Gamma = \{ \text{count} : \text{int} \}$

$$\frac{\frac{\Gamma(\text{count}) = \text{int}}{\Gamma \vdash \text{count} : \text{int}} \quad \frac{}{\Gamma \vdash 1 : \text{int}}}{\Gamma \vdash \text{count} + 1 : \text{int}}$$

- Each premise needs its own justification.
- A complete derivation (=a proof) of the judgment forms a tree.
(proof tree)

Ill-Typed Expressions

Goal: $\Gamma \vdash 1 + \text{true} : \text{int}$

$$\frac{\frac{}{\Gamma \vdash 1 : \text{int}} \quad \frac{?}{\Gamma \vdash \text{true} : \text{int}}}{\Gamma \vdash 1 + \text{true} : \text{int}}$$

- The addition rule requires true to have type int.
- The literal rule gives true the type bool.
- No rule derives the required premise $\Gamma \vdash \text{true} : \text{int}$.

Note: Types Are Not Necessarily Unique

$\tau ::= \text{int} \mid \text{uint} \mid \text{bool} \mid \tau_1 \rightarrow \tau_2 \text{ where } \tau_1, \tau_2 \in \{ \text{int}, \text{bool} \}$

$e ::= n \mid \text{true} \mid \text{false} \mid x$

$\mid -e \mid e_1 + e_2 \mid e_1 \&\& e_2 \mid e_1 == e_2 \mid e_1(e_2)$

$\Gamma \vdash n: \text{int}$

$\Gamma \vdash n: \text{uint}$

$$\frac{\Gamma \vdash e: \text{int}}{\Gamma \vdash -e: \text{int}}$$

Advanced Topic: *Coercion* Is a Language Choice

`1 + true`

- A language can define an implicit conversion before an operation.
- Our example language has no conversion from bool to int.
 - Its addition rule therefore rejects this expression.

A Typechecking Algorithm

Algorithmic Type Checking

`check( $\Gamma$ ,  $e$ ,  $ty$ ) : bool`

- Goal: To implement a function `check()` that returns true only when the typing judgment holds:

$$\forall \Gamma, e, \tau. \text{check}(\Gamma, e, \tau) = \text{true} \Rightarrow \Gamma \vdash e : \tau$$

- The converse does not have to be true.

Idea: Selecting a Rule

E.g., Goal: $\Gamma \vdash \text{count} + 1 : \text{int}$

$$\frac{\Gamma \vdash \text{count} : \text{int} \quad \Gamma \vdash 1 : \text{int}}{\Gamma \vdash \text{count} + 1 : \text{int}}$$

- The outer form of the expression selects the ADD rule.
- Its premises become two smaller proof goals.

Idea: Discharging Subgoals

check(Γ , count + 1, int)
 check(Γ , count, int)
 $\Gamma(\text{count}) = \text{int} ?$
 check(Γ , 1, int)
 true

$$\frac{\Gamma \vdash \text{count} : \text{int} \quad \Gamma \vdash 1 : \text{int}}{\Gamma \vdash \text{count} + 1 : \text{int}}$$

check(Γ , count + 1, int) should check two judgments:

- The first subgoal succeeds if $\Gamma(\text{count}) = \text{int}$.
- The second subgoal is unconditionally justified by the literal rule.

Checking the Basic Cases

$\text{check}(\Gamma, n, \tau) = (\tau = \text{int})$

$\text{check}(\Gamma, \text{true}, \tau) = (\tau = \text{bool})$

$\text{check}(\Gamma, \text{false}, \tau) = (\tau = \text{bool})$

$\text{check}(\Gamma, x, \tau) = (\text{lookup}(\Gamma, x) = \tau)$

$\text{check}(\Gamma, e_1 + e_2, \tau) =$

$(\tau = \text{int}) \ \&\&$

$\text{check}(\Gamma, e_1, \text{int}) \ \&\& \ \text{check}(\Gamma, e_2, \text{int})$

- An unbound variable makes checking fail.
- For conjunction, check both operands against bool.

Problem: Equality Needs the Operand Type

$$\Gamma \vdash e_1 : \sigma \quad \Gamma \vdash e_2 : \sigma \quad \sigma \in \{\text{int}, \text{bool}\}$$

$$\Gamma \vdash e_1 == e_2 : \text{bool}$$

```
check( $\Gamma$ ,  $e_1 == e_2$ ,  $\tau$ )  
  ( $\tau = \text{bool}$ ) &&  
  check( $\Gamma$ ,  $e_1$ , ???) && check( $\Gamma$ ,  $e_2$ , ???)
```

- The result type `bool` does not determine σ .
- How can we obtain the type of each operand?

Type Synthesis (= Inference)

$\text{synth}(\Gamma, e) : \text{type}$

$\text{synth}(\Gamma, e) = \tau \Rightarrow \Gamma \vdash e : \tau$

- Synthesis returns a type or reports a type error.

$\text{synth}(\Gamma, n) = \text{int}$

$\text{synth}(\Gamma, \text{true}) = \text{bool}$

$\text{synth}(\Gamma, \text{false}) = \text{bool}$

$\text{synth}(\Gamma, x) = \text{lookup}(\Gamma, x)$

What if lookup fails?

Type Synthesis: Inductive Cases

```
synth( $\Gamma$ ,  $e_1 == e_2$ ):  
   $\sigma_1 = \text{synth}(\Gamma, e_1)$   
   $\sigma_2 = \text{synth}(\Gamma, e_2)$   
  require  $\sigma_1 = \sigma_2$  and  $\sigma_1 \in \{\text{int}, \text{bool}\}$   
  return bool
```

```
synth( $\Gamma$ ,  $f(a)$ ):  
  require synth( $\Gamma$ ,  $f$ ) has the form  $\sigma \rightarrow \tau$   
  require check( $\Gamma$ ,  $a$ ,  $\sigma$ ) = true  
  return  $\tau$ 
```

Note: synth() uses check()

- Typechecking may use more information.
- Typechecking may succeed when synthesis fails.

Bidirectional Type Checking

$$\Gamma \vdash e_1 : \sigma \quad \Gamma \vdash e_2 : \sigma \quad \sigma \in \{\text{int}, \text{bool}\}$$

$$\Gamma \vdash e_1 == e_2 : \text{bool}$$

```
check( $\Gamma$ ,  $e_1 == e_2$ ,  $\tau$ )
```

```
  ( $\tau = \text{bool}$ ) &&
```

```
  let  $\tau_1 = \text{synth}(\Gamma, e_1)$  in
```

```
  let  $\tau_2 = \text{synth}(\Gamma, e_2)$  in
```

```
  ( $\tau_1 = \tau_2$ ) && ( $\tau_1 \in \{\text{int}, \text{bool}\}$ ) &&
```

```
  check( $\Gamma, e_1, \tau_1$ ) && check( $\Gamma, e_2, \tau_2$ )
```

Note: check() uses synth()
- “mutual recursion”



The Fallback

```
check( $\Gamma$ , e, expected):  
    let actual = synth( $\Gamma$ , e) in  
    return (actual = expected)
```

- This fallback is always sound.
- Not for every case; `synth()` may not choose the expected type.

E.g., in RustY:

```
let x : i32 := 42;
```

```
let x : usize := 42;
```

What Does the Algorithm Establish?

$\text{check}(\Gamma, e, \tau) = \text{true} \Rightarrow \Gamma \vdash e : \tau$

$\text{synth}(\Gamma, e) = \tau \Rightarrow \Gamma \vdash e : \tau$

- These are soundness claims about the algorithm.